Name: Dhruv Jain
Roll No: 2301045
Batch: T12

Experiment 3

Aim: To perform various GIT operations on local and remote repositories using GIT Cheat-Sheet.

Theory:

1. The Git Workflow Architecture

The Git environment is divided into three main local sections and one remote section. Understanding these is crucial for mastering the lifecycle of a file.

- Working Directory: The actual files on the local disk where modifications are made.
- Staging Area (Index): A technical "pre-commit" area where changes are gathered before being recorded.
- Local Repository (.git): The hidden directory where Git stores the metadata and object database for the project history.
- Remote Repository: A version of the project hosted on the internet or a network (e.g., GitHub, GitLab) that allows for collaboration.

2. Local Repository Operations

Local operations are performed entirely on the user's machine without the need for an internet connection.

- Initialization (git init): This command creates a new sub-directory named .git that contains all necessary repository files. It transforms a standard directory into a Git repository.
- Staging (git add): The staging process selects specific changes from the working directory to be included in the next snapshot. This allows for granular control over what is committed.
- Committing (git commit): A commit serves as a snapshot of the project at a specific point in time. Each commit is identified by a unique SHA-1 hash and includes a descriptive message.
- Status and History (git status, git log): The status command provides the current state of the working directory and staging area, while the log command displays the chronological history of commits.

3. Branching and Merging

Branching is the process of creating a separate line of development. The default branch is typically named main or master.

- Branch Creation: Creating a branch allows for experimental features or bug fixes without affecting the stable code.
- Merging: Once development on a branch is complete, it is integrated back into the primary branch. If the same lines of a file were modified in both branches, a merge conflict occurs, requiring manual resolution.

4. Remote Repository Operations

Remote operations facilitate the synchronization of data between the local machine and a central server.

- Cloning (git clone): This command creates a local copy of an existing remote repository, including all files, branches, and commit history.
- Remote Tracking (git remote): This manages the set of tracked repositories. The default remote name is usually origin.
- Pushing (git push): This operation uploads local repository content to a remote repository. It transfers commits from the local branch to the corresponding remote branch.
- Fetching vs. Pulling:
  - Fetch (git fetch): Downloads updates from the remote repository but does not integrate them into the local working files.
  - Pull (git pull): A combination of git fetch and git merge. It downloads the data and immediately attempts to integrate it into the current local branch.

5. Git Command Reference (Cheat-Sheet Summary)

Category	Command	Description
Setup	git init	Initializes a local repository.
	git clone [url]	Downloads a project and its history.
Local Changes	git add [file]	Adds a file to the staging area.
	git commit -m "[msg]"	Records the staged snapshot permanently.
Tracking	git status	Lists files to be committed or staged.
	git log	Shows the commit history.
Remote	git push [remote] [branch]	Uploads local commits to the remote.
	git pull	Fetches and merges remote changes.

Branching	git branch [name]	Creates a new branch.
	git checkout [name]	Switches to a different branch.

Implementation:

mkdir

```
202@LAB07-012 MINGW64 ~ (main)
$ mkdir git-new
```

cd

```
202@LAB07-012 MINGW64 ~ (main)
$ cd git-new
```

Global

```
202@LAB07-012 MINGW64 ~/git-new (main)
$ git config --global
error: no action specified
```

Global with username and email

```
202@LAB07-012 MINGW64 ~/git-new (main)
$ git config --global user.name "aryan"

202@LAB07-012 MINGW64 ~/git-new (main)
$ git config --global user.email "dlcatsec@gmail.com"

202@LAB07-012 MINGW64 ~/git-new (main)
$ git config --global --list
user.name=aryan
user.email=dlcatsec@gmail.com
```

```
202@LAB07-012 MINGW64 ~/git-new (main)
$ cat ~/.gitconfig
[user]
        name = aryan
        email = dlcatsec@gmail.com
```

init

```
202@LAB07-012 MINGW64 ~/git-new (main)
$ mkdir git-demo-project

202@LAB07-012 MINGW64 ~/git-new (main)
$ cd git-demo-project

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (main)
$ git init
Initialized empty Git repository in C:/Users/202/git-new/git-demo-project/.git/
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ ls -a
./  ../  .git/
```

add and commit

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git add .

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git commit -m "First Commit"
[master (root-commit) a987b66] First Commit
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git add .

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git commit -m "Second Commit"
[master 0a35c35] Second Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
```

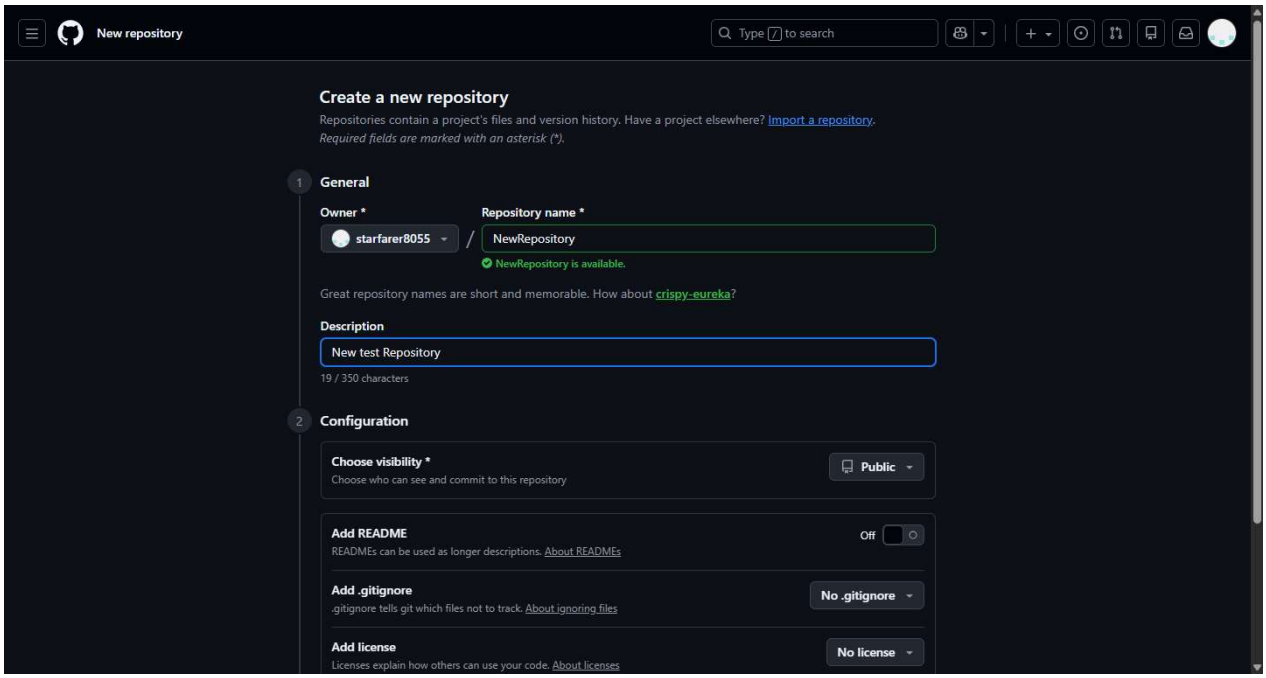
```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ nano index.html

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ |
```

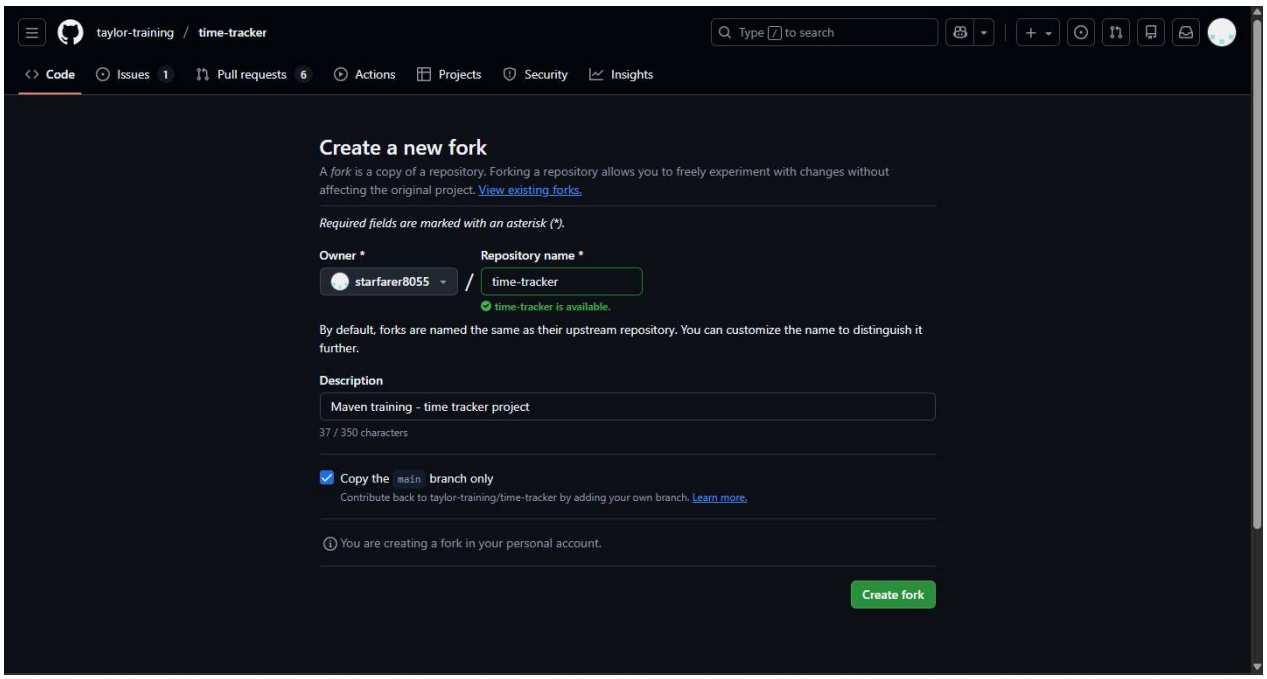
```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git commit -am "Express Commit"
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
[master c06c7e0] Express Commit
 1 file changed, 11 insertions(+)
```

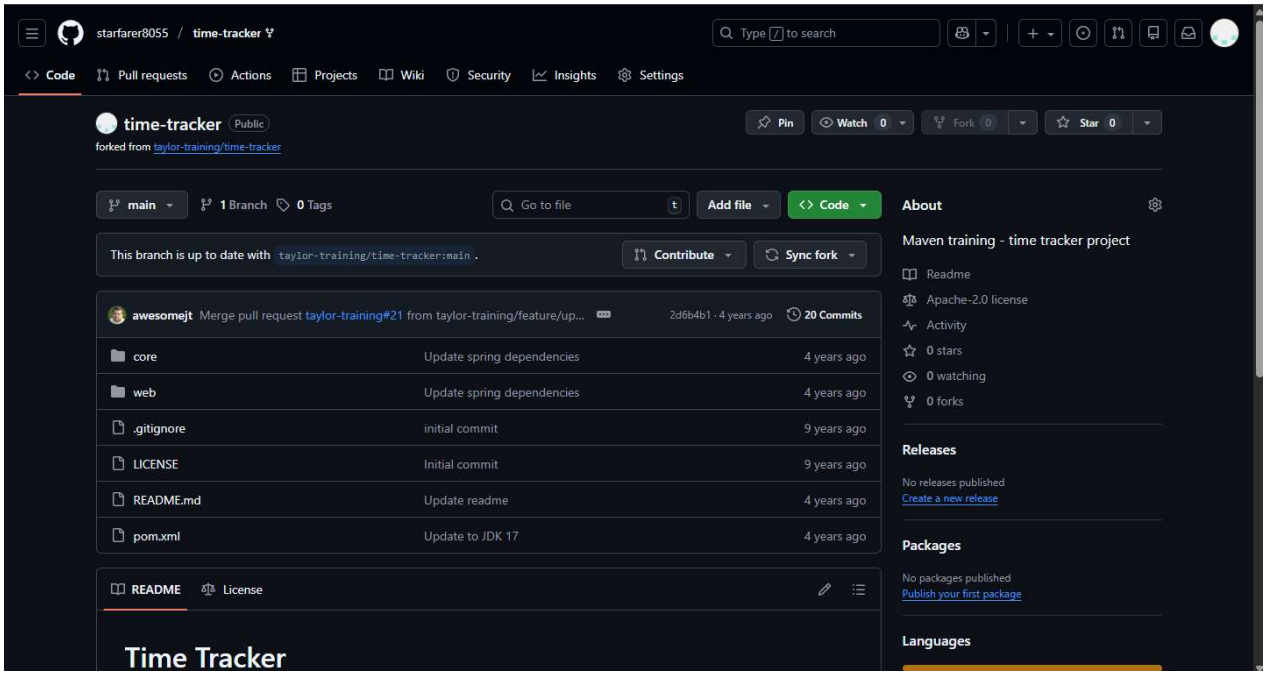
log

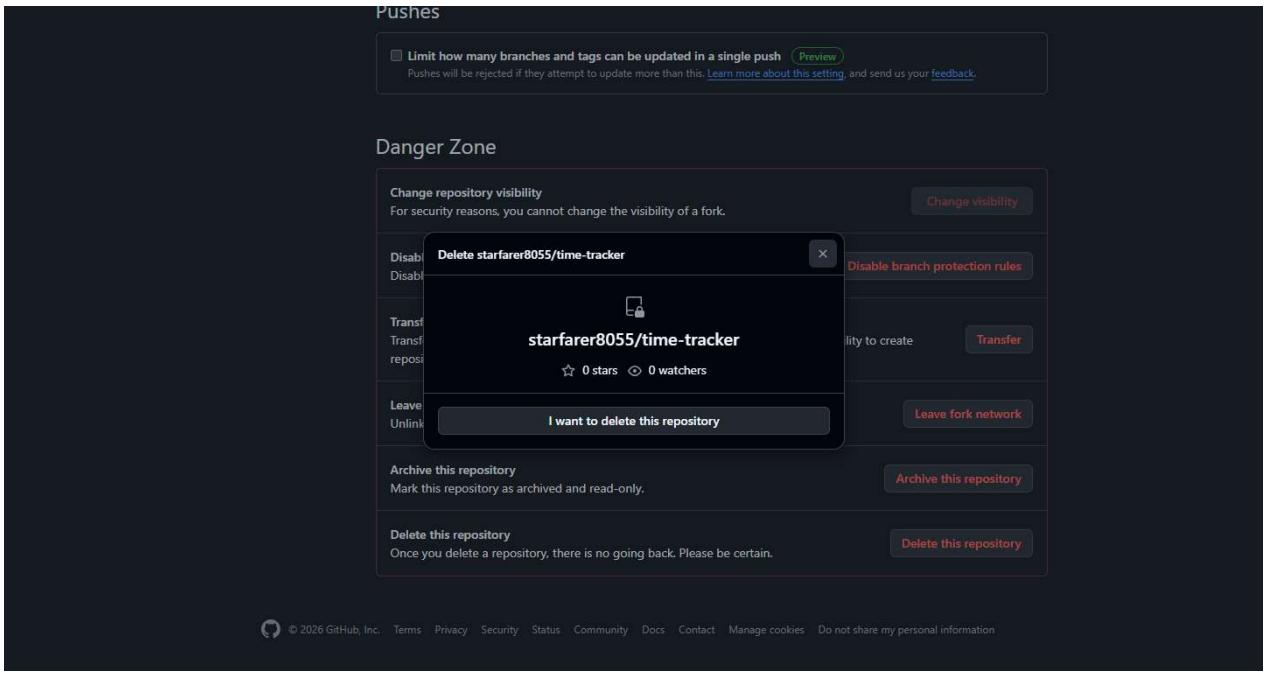
```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git log
commit c06c7e08bea48e457f1277ea83e8a62c49062e40 (HEAD -> master)
Author: aryan <dlcatsec@gmail.com>
Date:   Tue Jan 13 15:12:31 2026 +0530

    Express Commit

commit 0a35c35574edeb20af409afd0917cd93ad217d02
Author: aryan <dlcatsec@gmail.com>
Date:   Tue Jan 13 15:05:41 2026 +0530

    Second Commit

commit a987b66acc22f6dec59cf574f5d9e9c94102485c
Author: aryan <dlcatsec@gmail.com>
Date:   Tue Jan 13 14:59:59 2026 +0530

    First Commit
```

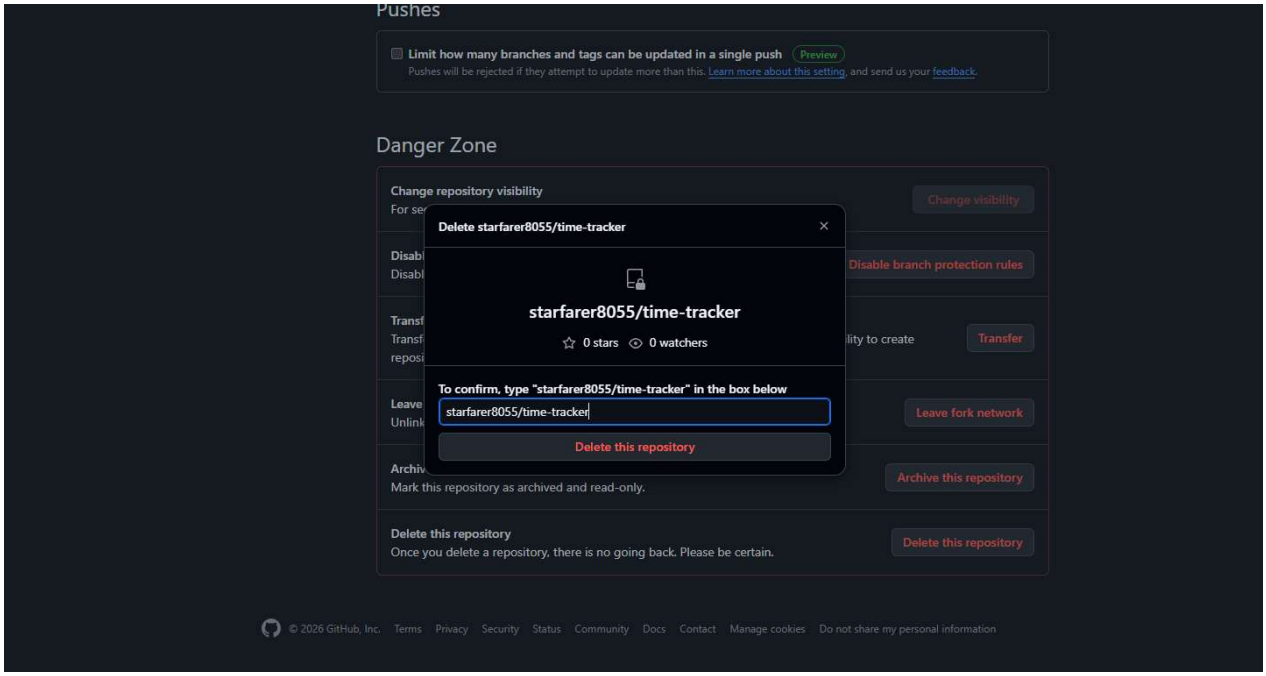
GitHub




taylor-training / time-tracker
Type / to search
Code
Issues 1
Pull requests 6
Actions
Projects
Security
Insights
Create a new fork
A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. View existing forks.
Required fields are marked with an asterisk (*).
Owner *
Repository name *
starfarer8055
time-tracker
time-tracker is available.
By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.
Description
Maven training - time tracker project
37 / 350 characters
Copy the main branch only
Contribute back to taylor-training/time-tracker by adding your own branch. Learn more.
You are creating a fork in your personal account.
Create fork


starfarer8055 / time-tracker
Type / to search
Code
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
time-tracker Public
forked from taylor-training/time-tracker
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
This branch is up to date with taylor-training/time-tracker:main.
Contribute
Sync fork
awesomejt Merge pull request taylor-training#21 from taylor-training/feature/up...
2d6b4b1 · 4 years ago
20 Commits
core
Update spring dependencies
4 years ago
web
Update spring dependencies
4 years ago
.gitignore
initial commit
9 years ago
LICENSE
Initial commit
9 years ago
README.md
Update readme
4 years ago
pom.xml
Update to JDK 17
4 years ago
README
License
Time Tracker
About
Maven training - time tracker project
Readme
Apache-2.0 license
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Languages






```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git clone https://github.com/AryanRajendraDalvi/profile.git
Cloning into 'profile'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
Resolving deltas: 100% (1/1), done.
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ ls
NewRepository/  index.html  profile/  test.txt
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git remote add origin https://github.com/starfarer8055/NewRepository.git

202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/starfarer8055/NewRepository.git
  Push  URL: https://github.com/starfarer8055/NewRepository.git
  HEAD branch: (unknown)
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 847 bytes | 847.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/starfarer8055/NewRepository.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 945 bytes | 42.00 KiB/s, done.
From https://github.com/starfarer8055/NewRepository
   c06c7e0..f5972ba  master     -> origin/master
Updating c06c7e0..f5972ba
Fast-forward
 test.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git log --oneline origin/master
f5972ba (HEAD -> master, origin/master, origin/HEAD) Third Commit
c06c7e0 Express Commit
0a35c35 Second Commit
a987b66 First Commit
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 950 bytes | 63.00 KiB/s, done.
From https://github.com/starfarer8055/NewRepository
   f5972ba..ef974a1  master     -> origin/master
```

```
202@LAB07-012 MINGW64 ~/git-new/git-demo-project (master)
$ git merge origin/master
Updating f5972ba..ef974a1
Fast-forward
 test.txt | 1 +
 1 file changed, 1 insertion(+)
```

Conclusion:

Hence, we have successfully performed various GIT operations on local and remote repositories using GIT Cheat-Sheet.

⌘⌘⌘⌘⌘⌘⌘